

Project Phoenix → CY015 Authentication Attack Test Cases

1. Overview

This document is about the security tests which I run on the Phoenix login system. The main goal was to check if the authentication side of the system is secure or not by trying different types of attacks on it. I tested 5 different attack scenarios and wrote down exactly what I did, what I expected to happen and what actually happened. Everything is linked back to the threat analysis work done earlier in CY005 so it all connects together properly.

2. Endpoints Tested

These are the API endpoints I tested. Think of endpoints like doors into the system — we are checking if those doors are properly locked.

Method	Endpoint	What It Does
POST	/api/users/auth/login	The main login door checks username and password
POST	/api/users/auth/register	Creates a new user account
POST	/api/users/auth/refresh	Gets a fresh access token when the old one expires
POST	/api/users/auth/logout	Logs the user out and kills their session
GET	/api/users/threats	Returns threat intelligence data should only work if logged in

3. Test Environment

I set everything up on my own machine before starting the tests. Below is a quick summary of what I used:

Field	Details
Local URL	http://localhost:3001
Backend Setup	Docker 6 containers were running which included the database, message queue, cache and all the main services
Tools Used	Postman, Python script, Burp Suite
Test Date	May 2026

Field	Details
Tester	Himanshu Khanna — Sub Team Leader CY015

4. Test Cases

AT-01 → Brute Force Attack

So basically, what I was trying to do here was keep sending wrong passwords to the login page over and over to see if the system would eventually stop me or just keep letting me try.

Field	Details
Test ID	AT-01
STRIDE ID	ST-03
Endpoint	POST /api/users/auth/login
Tool Used	Python Script

What I did:

1. I had a Python script ready for this, just needed to update the URL to <http://localhost:3001> and fix the username field name to match what the backend actually expects
2. Ran it from PowerShell
3. The script fired off 15 login requests one after another, all with the wrong password using the real test account analyst_test1
4. I was watching the output the whole time to see when or if the system would block me

```
Windows PowerShell
RECOMMENDATION: Implement rate limiting or account lockout on the login endpoint

Take a screenshot of this output for your test evidence!
=====
PS C:\Users\HP> notepad E:\Attack\brute_force_test.py
PS C:\Users\HP> python E:\Attack\brute_force_test.py
=====
Phoenix CY015 - AT-01 Brute Force Test
Endpoint: http://localhost:3001/api/users/auth/Login
=====
Attempt 01: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 02: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 03: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 04: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 05: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 06: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 07: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 08: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 09: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 10: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 11: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 12: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 13: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 14: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 15: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
=====
TEST COMPLETE - SUMMARY
=====
RESULT: FAIL - System did NOT rate limit or lock account after 15 failed attempts
RECOMMENDATION: Implement rate limiting or account lockout on the login endpoint

Take a screenshot of this output for your test evidence!
=====
PS C:\Users\HP>
```

```
Windows PowerShell
=====
TEST COMPLETE - SUMMARY
=====
RESULT: FAIL - System did NOT rate limit or lock account after 15 failed attempts
RECOMMENDATION: Implement rate limiting or account lockout on the login endpoint

Take a screenshot of this output for your test evidence!
=====
PS C:\Users\HP> notepad E:\Attack\brute_force_test.py
PS C:\Users\HP> python E:\Attack\brute_force_test.py
=====
Phoenix CY015 - AT-01 Brute Force Test
Endpoint: http://localhost:3001/api/users/auth/Login
=====
Attempt 01: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 02: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 03: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 04: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 05: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 06: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 07: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 08: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 09: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 10: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 11: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 12: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 13: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 14: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
Attempt 15: Status 401 - OK - Rejected (401 Unauthorized) - no lockout yet
=====
TEST COMPLETE - SUMMARY
=====
RESULT: FAIL - System did NOT rate limit or lock account after 15 failed attempts
RECOMMENDATION: Implement rate limiting or account lockout on the login endpoint

Take a screenshot of this output for your test evidence!
=====
PS C:\Users\HP>
```

Field	Details
Expected	Should have blocked me after a few wrong tries — either with a 429 or by locking the account
Actual	Got 401 invalid username or password every single time for all 15 attempts. It never once stopped me or slowed down even with a real account
Result	FAIL
Fix Needed	Needs rate limiting on the login endpoint after about 5 wrong attempts it should start blocking

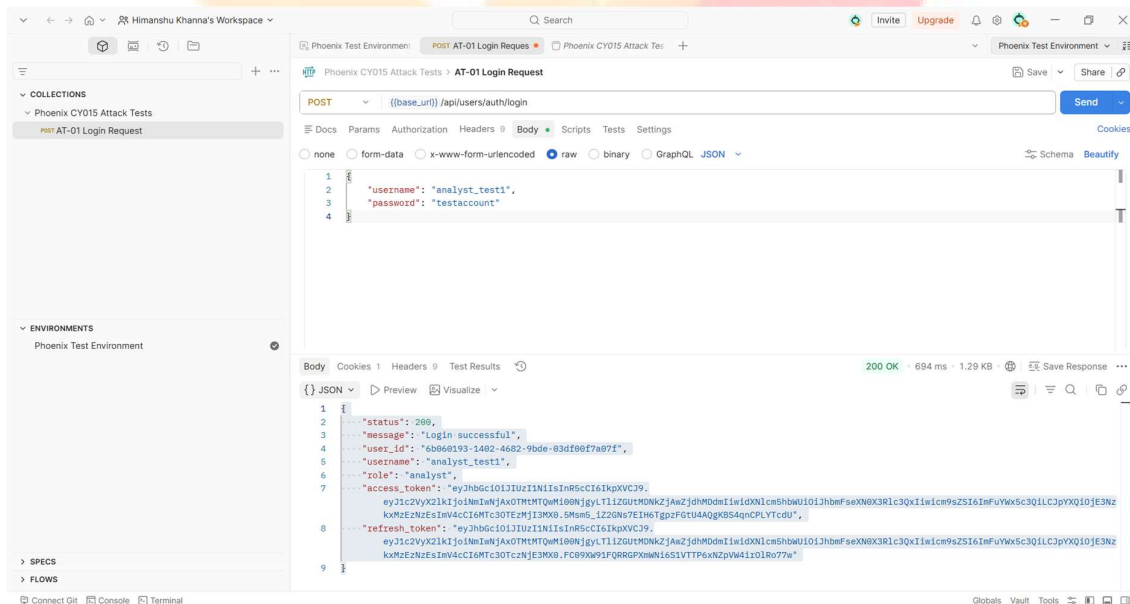
AT-02 → Token Replay After Logout

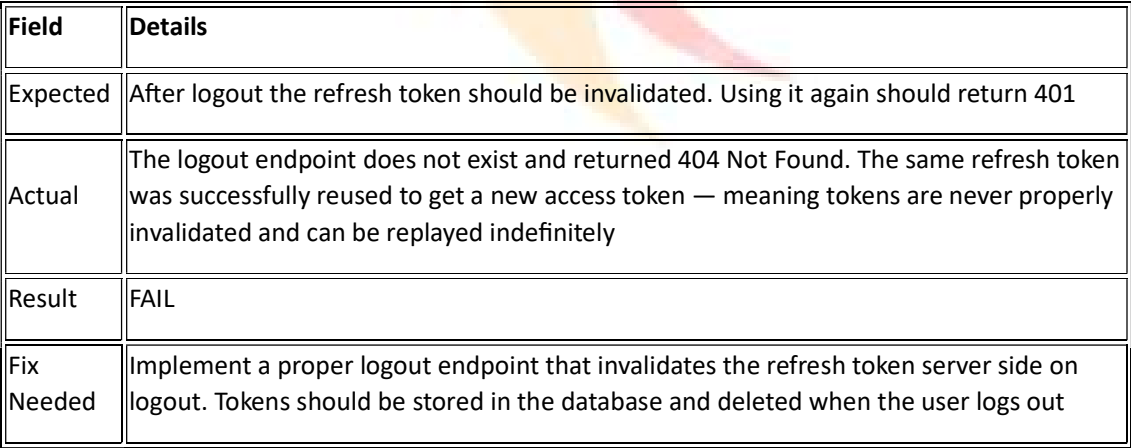
So the idea here was to check what happens if someone grabs a login token and tries to keep using it even after the original user has logged out. If the system does not properly kill tokens on logout then a stolen token stays valid forever which is a real problem.

Field	Details
Test ID	AT-02
STRIDE ID	ST-03
Endpoint	POST /api/users/auth/refresh
Tool Used	Postman

What I did:

1. Logged in with `analyst_test1` and got a real refresh token from the response
2. Tried to call the `logout` endpoint to end the session → `POST /api/users/auth/logout`
3. The `logout` endpoint returned `404 Not Found` → it does not exist on the backend yet
4. Then tried to use the same refresh token on the `refresh` endpoint to see if it still worked





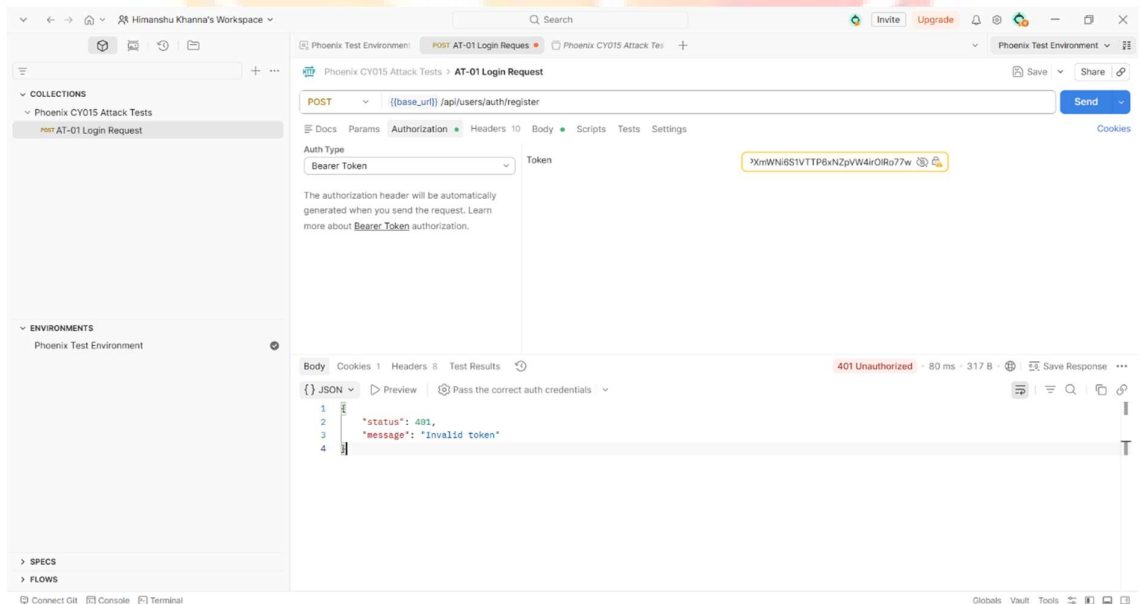
AT-03 → Privilege Escalation via Role Field

The idea here was to see if i could just put role admin in the signup request and get admin access. Some systems blindly accept whatever the client sends which would be a big problem.

Field	Details
Test ID	AT-03
STRIDE ID	ST-25
Endpoint	POST /api/users/auth/register
Tool Used	Postman

What I did:

1. First tried with an analyst token → set up a POST request to the register endpoint with role set to admin in the body
2. That returned 401 Invalid token → analysts cannot register accounts which is correct
3. Then logged in as admin1 and used the admin token in the Authorization header
4. Sent the same register request with role set to admin in the body



Admin Login

Field	Details
Result	Partial Pass — unauthenticated and non-admin users cannot register. However the system accepts the role value directly from the client which means an admin can create unlimited admin accounts by just sending role admin in the body
Fix Needed	The backend should never trust the role field sent by the client. Role assignment should always be handled server side only regardless of what the client sends

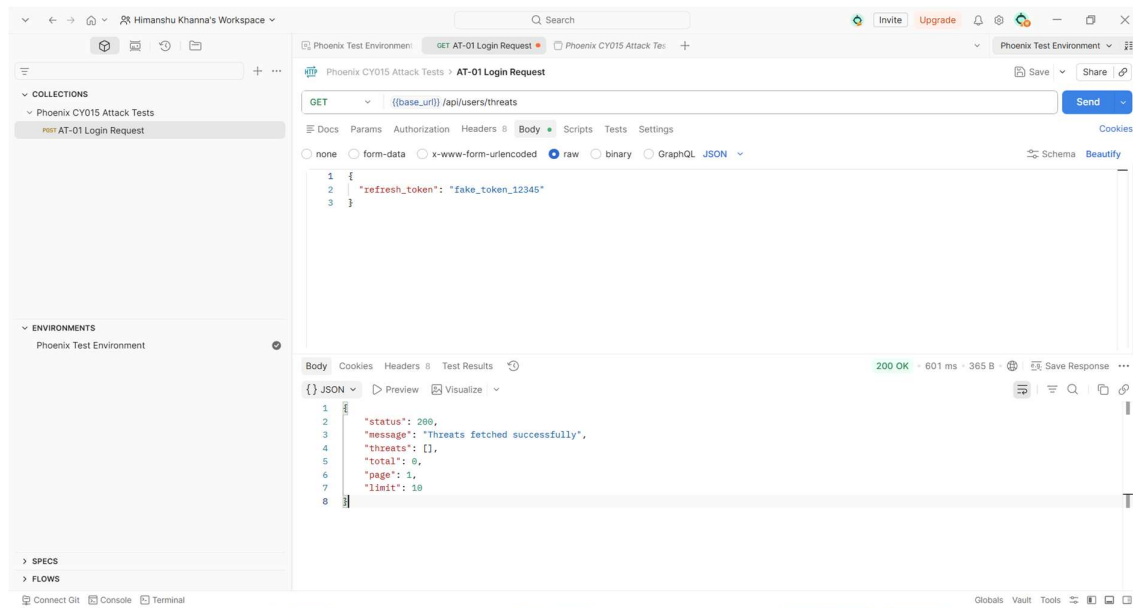
AT-04 → Unauthenticated Access to Threat Data

This was checking whether someone who is not logged in at all can still pull sensitive data from the system. Any endpoint that returns private data should refuse to work without a valid login token.

Field	Details
Test ID	AT-04
STRIDE ID	ST-17
Endpoint	GET /api/users/threats
Tool Used	Postman

What I did:

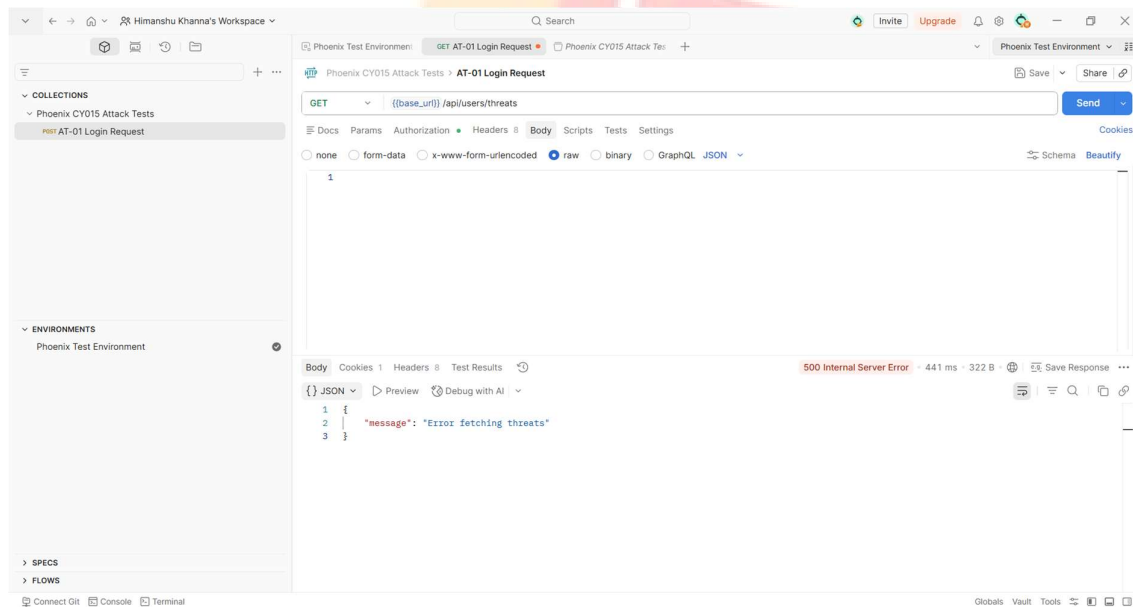
1. As I set up a GET request to the threats endpoint
2. Made sure there was absolutely nothing attached, no token, no auth, nothing
3. Sent the request and checked what came back



So basically, what happened was, the first time I tried this as I had no token, no login, nothing attached to the request at all. I just hit send and the system came back with 200 OK and said threats fetched successfully. That was the moment I realised oh wait this is actually a problem — anyone can just walk in and grab this data without even having an account.

Then later when I tried the same thing again it gave me a 500 error. But that is not because the system suddenly got secure or anything like that. It is just because the database had no data in it at that point so the system crashed trying to fetch something that was not there. The door was still open and there just happened to be nothing inside that time.

So the finding still stands where the endpoint never checked for a login token. It just let anyone in.



Field	Details
Expected	Should have refused with 401 since there was no login at all
Actual	Got 200 OK and the system sent back threat data without requiring any authentication token at all. Anyone without an account can access sensitive threat intelligence just by hitting the endpoint
Result	FAIL — CRITICAL
Fix Needed	This needs to be fixed urgently. Every endpoint that returns sensitive data must validate a Bearer token before returning anything. Authentication middleware needs to be applied to all data endpoints immediately

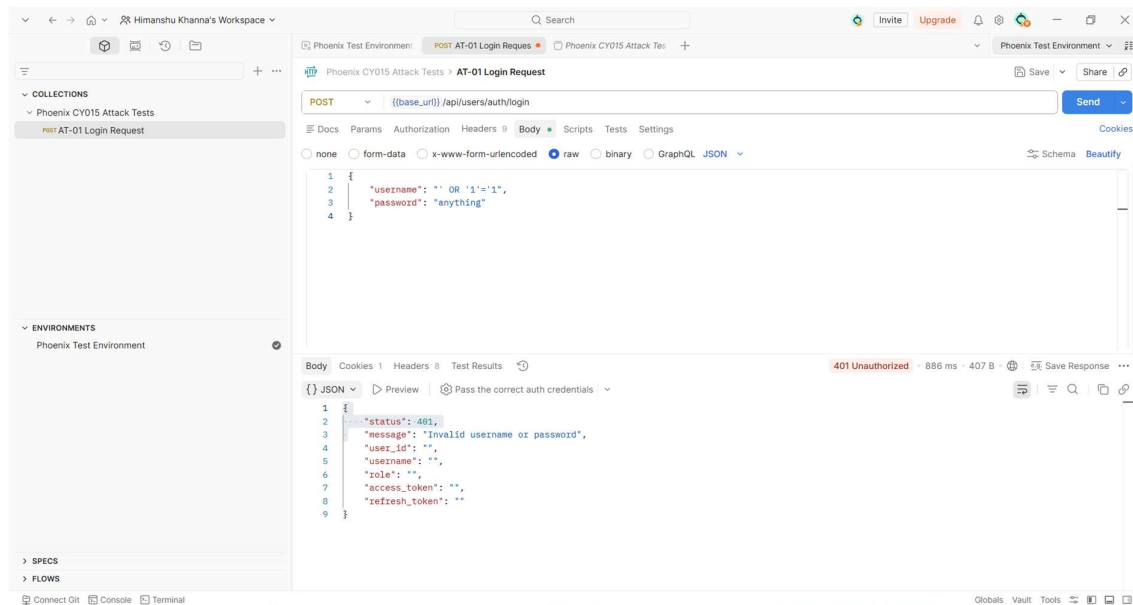
AT-05 → SQL Injection Attack

So basically what I was doing here was typing database commands into the login fields to see if the system would get confused and let me in. This is one of the oldest tricks in the book and a lot of systems still fall for it if input is not properly handled.

Field	Details
Test ID	AT-05
STRIDE ID	ST-10
Endpoint	POST /api/users/auth/login
Tool Used	Postman

What I did:

1. Set up a POST request to the login endpoint in Postman
2. In the username field I typed this — ' OR '1'='1 — which is a classic SQL injection payload that basically tells the database to ignore the password check
3. Put a random value in the password field
4. Hit send and watched to see if it would let me in



Field	Details
Expected	Should reject it straight away an injection like this should never result in a successful login
Actual	Got 401 — Invalid username or password. The payload was completely blocked and I did not get in
Result	PASS
Fix Needed	Nothing urgent right now, its handling it fine. Just make sure parameterised queries keep being used on the backend going forward

5. Summary of All Tests

Test ID	What We Tested	Tool	Result
AT-01	Brute Force Attack	Python	FAIL
AT-02	Token Replay After Logout	Postman	FAIL
AT-03	Privilege Escalation via Role Field	Postman	Partial Pass
AT-04	Accessing Data Without Login	Postman	FAIL — CRITICAL
AT-05	SQL Injection	Postman	PASS

6. Problems Found

Test	Problem	How Bad	What Needs to be Fixed
AT-01	No rate limiting on login endpoint	High	Add rate limiting — block after 5 wrong attempts
AT-02	Logout endpoint missing — tokens never invalidated	High	Implement logout endpoint that deletes refresh token server side
AT-03	Role field accepted from client during registration	Medium	Never trust role value from client — assign roles server side only
AT-04	Anyone can read threat data without logging in	Critical	Add authentication middleware to all data endpoints immediately

